

Tổng Hợp Kiến Thức

1. Tổng quan về API Routes

- **Mục đích:** Xử lý logic phía server, tương tác cơ sở dữ liệu, gọi API bên ngoài, xác thực người dùng, xử lý form submissions mà không cần backend riêng.
- **Lưu ý:** Chỉ nên dùng API Routes cho logic backend nhẹ. Với backend phức tạp, sử dụng các framework chuyên biệt như ExpressJS hoặc NestJS.
- **Vị trí:** Định nghĩa trong thư mục `app/api/` bằng các file `route.js`.

2. Route Handlers (Next.js 15, App Router)

- **Định nghĩa:** Tạo file `route.js` trong `app/api/` để xử lý các HTTP methods (GET, POST, PUT, DELETE, v.v.).
- **Cấu trúc:**

```
// app/api/hello/route.js
import { NextResponse } from 'next/server';

export async function GET(request) {
  return NextResponse.json({ message: 'Xin chào từ API!' });
}

export async function POST(request) {
  return NextResponse.json({ message: 'Đã nhận yêu cầu POST' });
}
```

- **Truy cập:** Kiểm tra bằng Postman tại `http://localhost:3000/api/hello`.
- **Lưu ý:** Không thể đặt `route.js` và `page.js` trong cùng một segment.

3. Làm việc với `NextRequest`

- **Query Parameters** (`searchParams`):

```
export async function GET(request) {
  const { searchParams } = request.nextUrl;
  const name = searchParams.get('name') || 'Khách';
  return NextResponse.json({ message: `Xin chào, ${name}!` });
}
```

- Kiểm tra: `http://localhost:3000/api/hello?name=Carol`.

- **Headers:**

```
import { headers } from 'next/headers';

export async function GET(request) {
  const headersList = headers();
  const userAgent = headersList.get('user-agent');
  return NextResponse.json({ userAgent });
}
```

- **Body** (POST, PUT, PATCH):

```
export async function POST(request) {
  const body = await request.json();
  return NextResponse.json({ receivedBody: body });
}
```

4. Tùy chỉnh `NextResponse`

- **Phương thức:**

- `NextResponse.json(body, init?)` : Trả về JSON.
- `NextResponse.redirect(url, init?)` : Chuyển hướng.
- `new NextResponse(body, init?)` : Tạo response tùy chỉnh.

- **Status Code:**

```
return NextResponse.json({ error: 'Không tìm thấy' }, { status: 404 });
```

- **Headers:**

```
const customHeaders = new Headers();
customHeaders.set('X-Custom-Header', 'Hello from Next.js API');
customHeaders.set('Content-Type', 'application/json');

const data = {
  message: 'Đây là phản hồi JSON',
  version: 'LetDiv 1.0',
  date: new Date().toISOString()
};

return new NextResponse(JSON.stringify(data), {
  status: 200,
  headers: customHeaders,
});
```

- **Chuyển hướng:**

```
export async function GET() {
  return NextResponse.redirect('https://nextjs.org/docs');
}
```

- **Cookies:**

```
export async function GET() {
  const response = NextResponse.json({ message: 'Cookie đã được thiết lập!' });
  response.cookies.set('myCookie', 'HelloNextjs', { path: '/', maxAge: 60 * 60 * 24 });
  return response;
}

export async function DELETE() {
  const response = NextResponse.json({ message: 'Cookie đã được xóa!' });
  response.cookies.delete('myCookie');
  return response;
}
```

- **HTTP Status Codes phổ biến:**

- `200 OK` : Thành công.
- `201 Created` : Tạo mới thành công (POST).
- `204 No Content` : Thành công, không trả nội dung (DELETE, PUT).
- `400 Bad Request` : Dữ liệu đầu vào lỗi.
- `401 Unauthorized` : Cần xác thực.
- `403 Forbidden` : Không có quyền.

- `404 Not Found` : Không tìm thấy tài nguyên.
- `500 Internal Server Error` : Lỗi server.

5. Dynamic API Routes

- **Cú pháp:** Sử dụng `[id]` hoặc `[...slug]` trong tên thư mục.
- **Ví dụ:**

```
// app/api/products/[id]/route.js
export async function GET(request, { params }) {
  const { id } = await params;
  return NextResponse.json({ message: `Sản phẩm ID: ${id}` });
}
```

```
// app/api/blog/[...slug]/route.js
export async function GET(request, { params }) {
  const { slug } = await params;
  return NextResponse.json({
    message: slug ? `Đường dẫn: /blog/${slug.join('/')}` : 'Trang blog chính',
    pathSegments: slug || [],
  });
}
```

6. Xử lý lỗi (Error Handling)

- **Tầm quan trọng:** Trả về phản hồi lỗi rõ ràng, tránh crash server, bảo mật thông tin nhạy cảm.
- **Cách làm:**

```
export async function GET() {
  try {
    throw new Error('Lỗi giả lập!');
    return NextResponse.json({ message: 'Thành công!' });
  } catch (error) {
    console.error('Lỗi:', error.message);
    return NextResponse.json(
      { message: 'Đã xảy ra lỗi', error: error.message, code: 'API_ERROR_001' },
      { status: 500 }
    );
  }
}
```

- **Lưu ý:** Không trả chi tiết lỗi (stack trace) trong môi trường production.